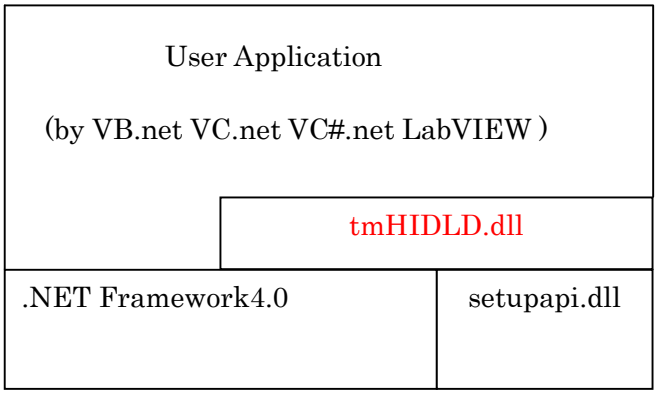
Pico second LD board control library manual

Tama Electric Inc. 2016/10/11

This manual describes how to use tmHIDLD.dll which controls LD board. The library works on PC which is installed .Net Framework 4.0 or later.



This USB control library has the following features.

- No need to install USB driver software. This library uses setupapi.dll which is installed in every Windows PC.
- It's easy to develop user own software with .NET Framework system.
- Single library can control multiple boards.

Operation systems

Windows XP with .Net Framework 4.0 or later.
Windows Vista with .Net Framework 4.0 or later.
Windows 7　32bit,　64bit with .Net Framework 4.0 or later.

Fig 1 is block diagram of relationship between the hardware and the data. PC and the board is connected with USB 2.0 full speed mode (12MHz). Since the library system uses HID (Human Interface Device) mode, we do not need the USB driver softoware.

Fig. 1 Hardware and software data.

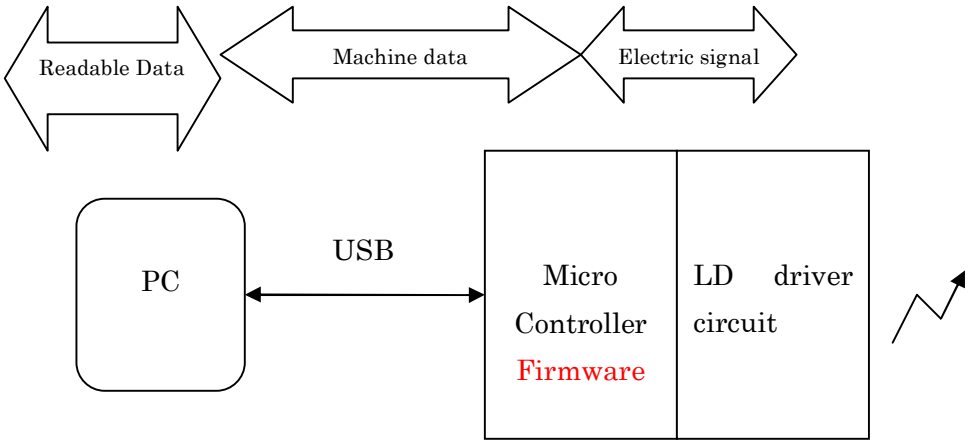
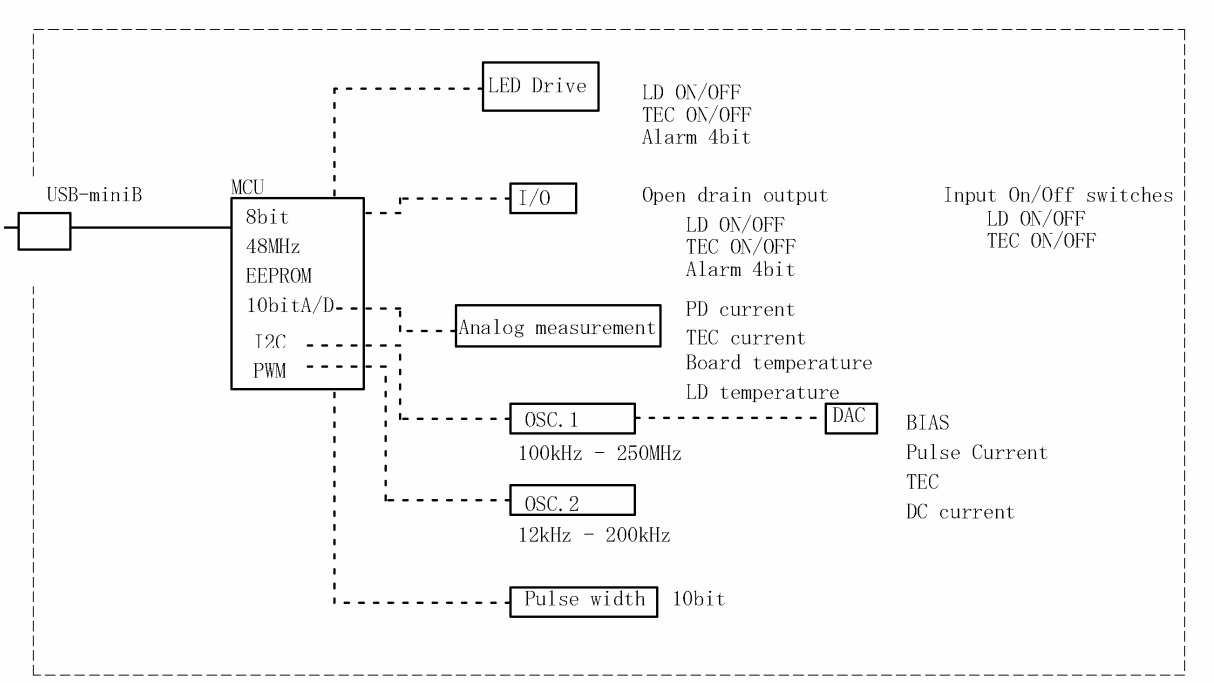
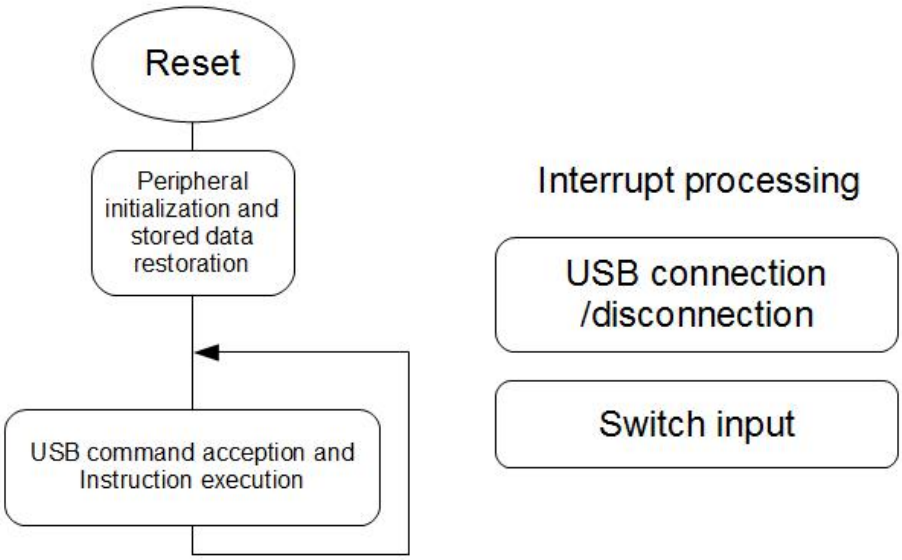


Fig. 2 Block diagram of hardware.



The operation of the firmware



When the power is turned ON, the microcomputer initialize peripheral devices and then restore the data stored in the EEPROM. Then it enters a loop that to receive instructions via USB and execute it. As an interrupt processing, there is a change of the USB connection state and a switch input. The USB connection state change interrupt is generated when the connector is connected and disconnected. When the USB is disconnected, it enters a standalone mode. At that time, if LD is ON or TEC is ON, it turns off both of them. There are LD ON and TEC ON in switch input.

Interrupt processing

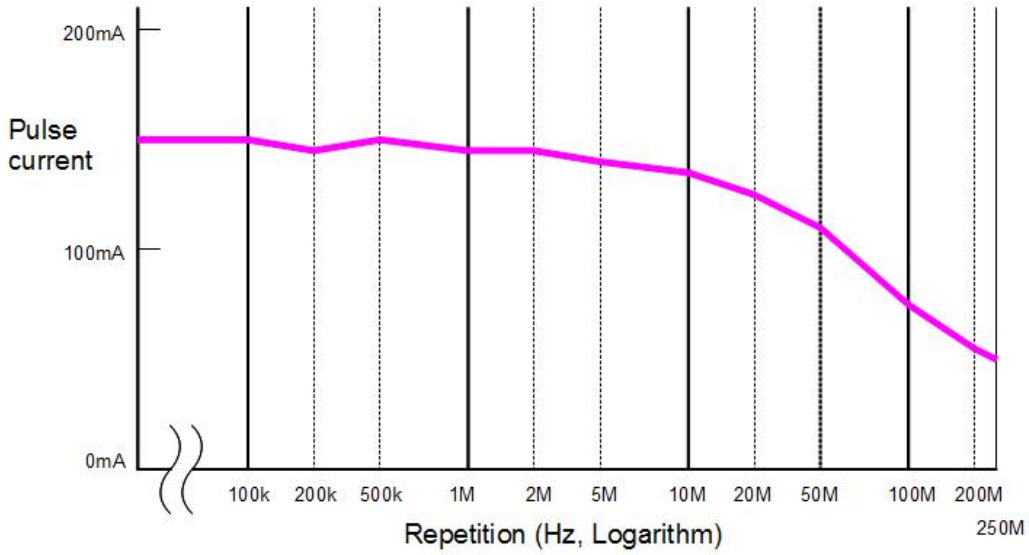
Items	Processing content
USB connected	Enters to USB mode. Turns off TEC and LD
USB disconnected	Enters to standalone mode. Turns off TEC and LD
LD switch ON or OFF	LD ON or OFF processing.
TEC switch ON of OFF	TEC ON or OFF processing.

Deference between USB mode and standalone mode

Mode	Processing content
USB	Control from PC via USB. LD ON and TEC ON instructions are AND operation with switches.
Standalone	Set by the USB is used. LD ON and TEC ON are work by switches.

About automatic light intensity correction (patent pending)

This pico second pulse laser light source has two oscillators' covers from 3 kHz to 250 MHz. Each repetition has a best pulse current setting for optimal wave form of light pulse. Automatic pulse current adjustment function will find the best pulse current when a repetition is set. The adjustment will be done with following characteristic of the LD.

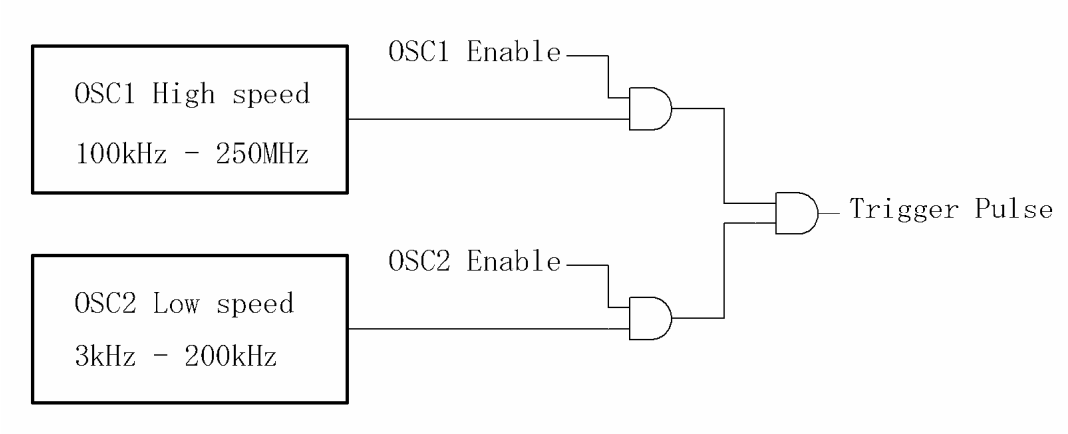


Characteristic example of the repetition and pulse current for optimal wave form.

In less than 100 kHz, the current value is the same value. According to the advance 200 kHz, 1 MHz, 10 MHz, 100 MHz and 250 MHz will be low current. Correction values are calculated by linear approximation between each point of the 100 kHz, 200 kHz, 500 kHz, 1 MHz, 2 MHz, 5 MHz, 10 MHz, 20 MHz, 50 MHz, 100 MHz, 200 MHz and 250 MHz. You can change the current value after it has been automatically corrected. Characteristic of the above diagram is different in each of the board.

Internal oscillator

Oscillator 1 will operate in the range of from 100 kHz to 250 MHz. Oscillator 2 will operate in the range of from 3 kHz to 200 kHz.



How to create the original program by Visual Basic

In the following, we will create the simple application. The program read a version of tmHIDLD.dll. Here we use Visual Studio 2010 Express. First of all, we must create a new project and copy tmHIDLD.dll file into the project file. We need add reference for tmHIDLD.dll. You can find the Add Reference dialog in a VB project by selecting the project node in Solution Explorer, right-clicking, and selecting Add Reference.

Next, we need create Loaded events by double-click the item on the Windows Forms Designer for which you wish to create a handler. And we need copy the followings code into the handler.

```
Dim usb As New tmHIDLD.tmHIDLD(True)
MsgBox(usb.getVersion.ToString())
```

Run the program by F5 then we will see the followings dialog.

For Visual C++

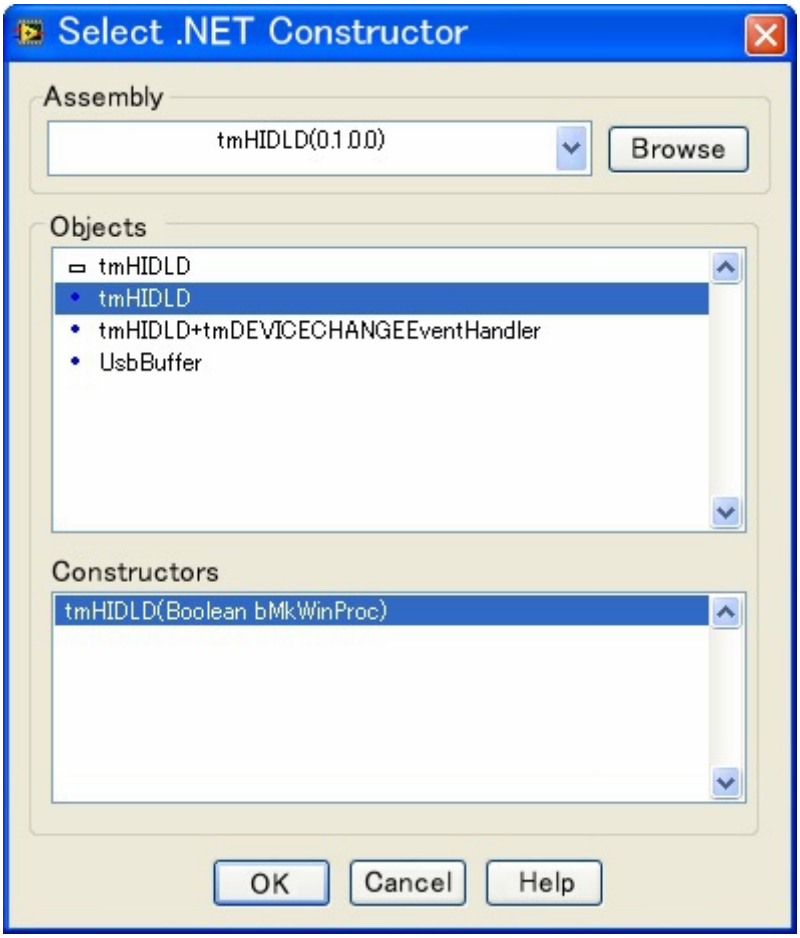
```
tmHIDLD::tmHIDLD^ usb = gcnew tmHIDLD::tmHIDLD(true);
MessageBox::Show(usb->getVersion());
```

For Visual C#

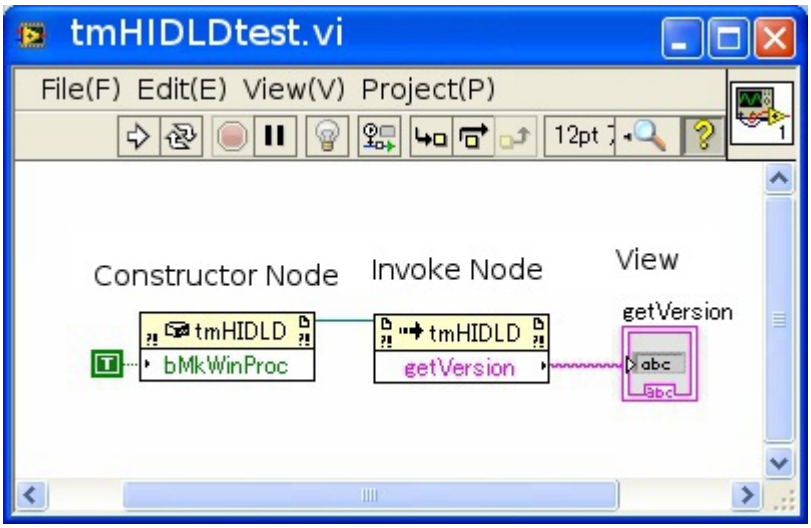
```
tmHIDLD.tmHIDLD usb = new tmHIDLD.tmHIDLD(true);
MessageBox.Show(usb.getVersion());
```

How to create the original program by LabVIEW

Place a Constructor Node on the block diagram to display this dialog box. Use this dialog box to create a .NET object.



Please set as left and click OK.



Please program as left.

Execution result is as follows.

If you have error message like followings when you create a '.NET Constructor', please see the follow link.
'The selected file is not a .NET assembly, type library or automation executable.'

http://digital.ni.com/public.nsf/allkb/32B0BA28A72AA87D8625782600737DE9

USB device control by the user program.

USB devices are connected and disconnected at any time by the user. Here, we will see how to handle these events by the user program.
tmHIDLD.dll will throw tmDEVICECHANGE event when USB is connected, disconnected and even other USB devices connected or disconnected. User program must call CheckUSB() function when get tmDEVICECHANGE event. If user program has any reason that can not get the event, it can use Updates property for polling sequence. User program can get tmDEVICECHANGE event signal when Updates property returns “True” value while running Updates property continuously. If user program read “True” value from Updates property, next value of Updates property is “False”. This function avoids signal overlapping.

Example of user program.

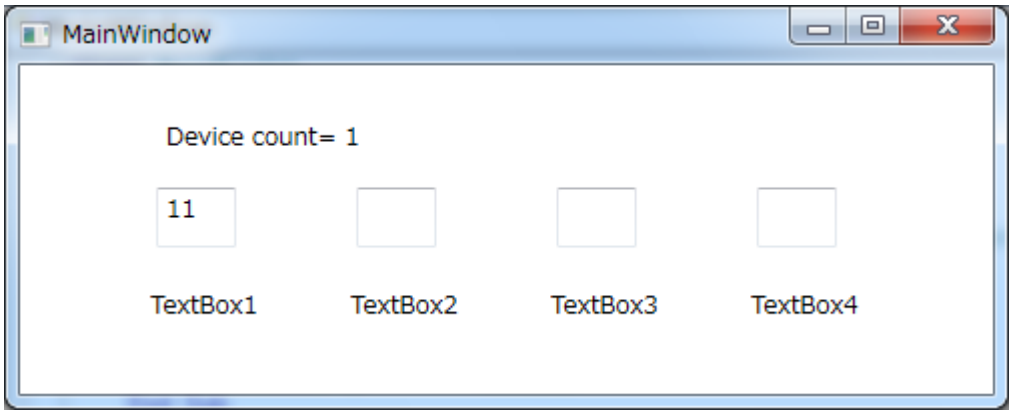
```
Class MainWindow
    'definition of event instance with events.
    Public WithEvents usblib As tmHIDLD.tmHIDLD

    'Initializing
    Private Sub Window_Loaded(ByVal sender As System.Object, _
                              ByVal e As System.Windows.RoutedEventArgs) _
                          Handles MyBase.Loaded
        usblib = New tmHIDLD.tmHIDLD(True)
        usblib.CheckUSB()    'First checking of USB

        UpdateUSBIF()        'Updating USB
    End Sub

    'Event handler of tmDEVICECHANGE
    Private Sub sDeviceChange(ByVal sender As Object, _
                              ByVal e As System.EventArgs) _
                         Handles usblib.tmDEVICECHANGE
        usblib.CheckUSB()    'Checking USB

        UpdateUSBIF()        'Updating USB
    End Sub

    Private Sub UpdateUSBIF()
        usblib.CheckUSB()     ' Checking USB

        'DevCount is number of connected board.
        Label1.Content = "Device count= " + usblib.DevCount.ToString

        Dim i As Integer
        TextBox1.Text = ""       'Clearing display
        TextBox2.Text = ""
        TextBox3.Text = ""
        TextBox4.Text = ""
        'CardNo(*) can read the card number of each board.
        For i = 0 To usblib.DevCount - 1
            If i = 0 Then TextBox1.Text = usblib.CardNo(i)
            If i = 1 Then TextBox2.Text = usblib.CardNo(i)
            If i = 2 Then TextBox3.Text = usblib.CardNo(i)
            If i = 3 Then TextBox4.Text = usblib.CardNo(i)
        Next

    End Sub
End Class
```

Example of Xaml file.

```
<Window x:Class="MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="MainWindow" Height="202" Width="502">
    <Grid Height="143" Width="450">
        <TextBox      Height="30"      HorizontalAlignment="Left"      Margin="50,50,0,0"
Name="TextBox1" VerticalAlignment="Top" Width="40" />
        <TextBox      Height="30"      HorizontalAlignment="Left"      Margin="150,50,0,0"
Name="TextBox2" VerticalAlignment="Top" Width="40" />
        <TextBox      Height="30"      HorizontalAlignment="Left"      Margin="250,50,0,0"
Name="TextBox3" VerticalAlignment="Top" Width="40" />
        <TextBox      Height="30"      HorizontalAlignment="Left"      Margin="350,50,0,0"
Name="TextBox4" VerticalAlignment="Top" Width="40" />
        <Label       Content="TextBox1"       Height="25"       HorizontalAlignment="Left"
Margin="42,96,0,0" VerticalAlignment="Top" />
        <Label       Content="TextBox2"       Height="25"       HorizontalAlignment="Left"
Margin="142,96,0,0" VerticalAlignment="Top" />
        <Label       Content="TextBox3"       Height="25"       HorizontalAlignment="Left"
Margin="242,96,0,0" VerticalAlignment="Top" />
        <Label       Content="TextBox4"       Height="25"       HorizontalAlignment="Left"
Margin="342,96,0,0" VerticalAlignment="Top" />
        <Label   Content="Device   count=   0"   Height="28"   HorizontalAlignment="Left"
Margin="50,12,0,0" Name="Label1" VerticalAlignment="Top" />
    </Grid>
```

Execution result of the program.

A card number of connected board is 11.



All functions and properties

MaxUnit Property [Integer]

It specifies the maximum number of board to be used.

UseUnit Property [Integer]

It specifies the number of board to be used.

Updates Property [Boolean, Read only]

It becomes true when tmDEVICECHANGE is thrown. It becomes false after reading. This property is used polling sequence in user program which can not use event handler. If user program can use event handler, it is better to catch tmDEVICECHANGE directory.

CardNo(n) Property [Integer, Read only]

n: The serial number of the board that starts from zero.

User program must run CheckUSB function before read the card number. It returns card number of connected board. If several boards was connected for example card number 1, 2, 6 and 8, then you get a result as follows.

CardNo(0) = 1

CardNo(1) = 2

CardNo(2) = 6

CardNo(3) = 8

DevCount Property [Integer, Read only]

It returns a number of connected boards.

getVersion Function(Void) [String]

It returns a string version of this tmHIDLD.dll.

SerialNumber(n) Property [8 elements Byte-type array]

n: The serial number of the board that starts from zero.

It read and writes 8 serial numbers as 8 elements Byte-type array. The warning message will be displayed as writing.

getFirmVersion(n) Function(Void) [String]

n: The serial number of the board that starts from zero.

It returns firmware version of the specified board. We can specify the board using n. Return strings is hexadecimal string format as 'FF FF'.

GetEEPROM(n, add, cnt) Function [Byte-type array]

n: The serial number of the board that starts from zero.

add: Address to read from. [0 to 255]

cnt: Number of data to read from. [0 to 64]

It read maximum 64 data from specified address of EEPROM as Byte-type array.

SetEEPROM(n, add, cnt, dat, mes) Function [Boolean]

n: The serial number of the board that starts from zero.

add: Address to write to. [0 to 255]

cnt: Number of data to write to. [0 to 64]

dat: Data to write to. [Byte-type array]

mes: Show warning or not. [Boolean]

It writes maximum 32 data to specified address of EEPROM. If a number of 'dat' is less than 'cnt', all of the data will not be written.

It returns Boolean-type data 'true' as writing successful or 'false' as not.

LDOnOff(cmd, n) Function [Void]

cmd: 0 as off, 1 as on.

n: The serial number of the board that starts from zero.

It turns on or off the LD. There is no return value.

TECOnOff(cmd, n) Function [Void]

cmd: 0 as off, 1 as on.

n: The serial number of the board that starts from zero.

It turns on or off the temperature control. There is no return value.

GetPD(n) Function [Single]

n: The serial number of the board that starts from zero.

It returns monitor photo diode current as Single-type. The unit is Pico Amps.

In addition, to load the following data from the board to PC.

We can be obtained the following information without having to USB communication many times. But please notice that if another program ran GetPD function for another board the following data will be changed to information of another board. All the following information is read only.

Parameter	Property name	Type	Unit
LD on / off	LDOn	Boolean	None
TEC on / off	TECOn	Boolean	None
Alarm signal 0	GetAlarm0	Boolean	None
Alarm signal 1	GetAlarm1	Boolean	None
Alarm signal 2	GetAlarm2	Boolean	None

Alarm signal 3	GetAlarm3	Boolean	None
LD Temperature	LD_Temp	Single	Degrees C
Board Temperature	BD_Temp	Single	Degrees C
TEC current	TEC_current	Single	1000=3.5A
TEC current (LDB-80)	TEC_current80	Single	183=1A
Bias current	Bias	Single	Milli Amps
Pulse current	Pulse	Single	Milli Amps
SOA current	SOA_current	Single	Milli Amps
PD current	PD_current	Single	Pico Amps
PD current(LDB-80)	PD_current80	Single	Pico Amps
Pulse width	PulseWidthB	Single	Pico Second
IO information	IOInfo	Byte	None
Resistance value of potentiometer	RV1Value	Single	%

SetTemp(n, temp) Function [Single]

n: The serial number of the board that starts from zero.

temp: Preset temperature from 0 degrees C to 40 degrees C.

0 degrees C to 40 degrees C for tmHIDLD40.dll

0 degrees C to 60 degrees C for tmHIDLD60.dll

It set LD temperature. Set temperature can be set in increments of 0.1 degrees C.

It returns preset temperature value as Single-type. The unit is degrees C.

GetTemp(n) Function [Single]

n: The serial number of the board that starts from zero.

It returns preset temperature value as Single-type. The unit is degrees C.

GetLdTemp(n) Function [Single]

n: The serial number of the board that starts from zero.

It returns measured LD temperature as Single-type. The unit is degrees C.

SetPGOnOff(n, PG1, PG2, Ext) Function [Void]

n: The serial number of the board that starts from zero.

PG1, PG2, Ext: ON/OFF [Boolean] True for On or false for Off.

It controls the pattern generator (Oscillator). It's possible to turn on both PG1 and PG2. If turn on Ext, both PG1 and PG2 will be turned off. There is no return value.

SetPG2SingleShot(n) Function [Void]

n: The serial number of the board that starts from zero.

It will raise the single pulse from PG2. We need to turn off PG2 before this single pulse. There is no return value.

GetPGOnOff(n) Function [Byte]

n: The serial number of the board that starts from zero.

It returns status of pattern generator as Byte-type data. The least significant bit of return value as binary data represents PG1, second bit is PG2, third bit is Ext. 1 represents on, 0 is off.

SetPG1Repetition(n, freq) Function [Void]

n: The serial number of the board that starts from zero.

freq: 0-250,000,000 [Uinteger]

It presets repetition value of PG1. The unit is Hz. There is no return value.

GetPG1Repetition(n) Function [Uinteger]

n: The serial number of the board that starts from zero.

It returns repetition value of PG1 as unsigned integer. The unit is Hz.

SetPG2Repetition(n, freq) Function [Void]

n: The serial number of the board that starts from zero.

freq: 0-200,000 [Uinteger]

It presets repetition value of PG2. The unit is Hz. There is no return value.

GetPG2Repetition(n) Function [Uinteger]

n: The serial number of the board that starts from zero.

It returns repetition value of PG2 as unsigned integer. The unit is Hz.

ImHere(n, sec) Function [Void]

n: The serial number of the board that starts from zero.

sec: [Integer] Will not be used.

It blinks the LED for confirmation. There is no return value.

SetBias(n, val) Function [Void]

n: The serial number of the board that starts from zero.

val: [Single] Bias current value from 0 to 200mA.

It preset the bias current value. The sum of the pulse and bias Please be careful so as not to exceed 200mA. There is no return value.

SetBiasAdj(n, val) Function [Void]

n: The serial number of the board that starts from zero.

val: [Single] Bias current value from 0 to 200mA.

It preset the bias current value without writing to the EEPROM.

GetBias(n) Function [Single]

n: The serial number of the board that starts from zero.

It returns the bias current value as Single-type data. The unit is mA.

SetLDCurrent(n, val) Function [Void]

n: The serial number of the board that starts from zero.

val: [Single] Pulse current value from 0 to 200mA.

It preset the pulse amplitude current value from 0 to 200mA. The sum of the pulse and bias Please be careful so as not to exceed 200mA. There is no return value.

SetLDCurrentAdj(n, val) Function [Void]

n: The serial number of the board that starts from zero.

val: [Single] Pulse current value from 0 to 200mA.

It preset the pulse amplitude current value without writing to the EEPROM. There is no return value.

GetLDCurrent(n) Function [Single]

n: The serial number of the board that starts from zero.

It returns the pulse amplitude current value as Single-type data. The unit is mA.

SetDCCurrent(n, val) Function [Void]

n: The serial number of the board that starts from zero.

val: [Single] DC current value for SOA from 0 to 300mA.

It presets DC current value. There is no return value.

SetDCCurrentAdj(n, val) Function [Void]

n: The serial number of the board that starts from zero.

val: [Single] DC current value for SOA from 0 to 300mA.

It preset DC current value without writing to the EEPROM. There is no return value.

GetDCCurrent(n) Function [Single]

n: The serial number of the board that starts from zero.

It returns the DC current value as Single-type data. The unit is mA.

HeaterMode(n, ICT) Function [Boolean]

n: The serial number of the board that starts from zero.

ICT: [Boolean] True for 30 degrees C or false for 84 degrees C.

It presets ICT mode. It controls temperature of IC which controls pulse width.

84 degrees C will be able to further stabilize the pulse width. It returns ICT mode as Boolean-type data.

PowerDownMode(n, PDN1) Function [Boolean]

n: The serial number of the board that starts from zero.

PDN1: [Boolean] True for On or false for Off.

It controls the power of analog circuit for LD current driver. It returns PDN1 mode as Boolean-type data.

AutoAdjustmentMode(n, NoAuto) Function [Boolean]

n: The serial number of the board that starts from zero.

NoAuto: [Boolean] True for non adjustment or false for adjustment.

It switches manual adjustment and automatic adjustment of pulse current. It returns NoAuto mode as Boolean-type data.

HeaterControl(n) Property [Byte]

n: The serial number of the board that starts from zero.

It gets or sets ICT, PDN1 and NoAuto.

List of setting bits from least significant bit as binary data is followings. More significant bits will be priority.

Bit0 : 1 for ICT is 30 degrees C or 0 for none.

Bit1 : 1 for ICT is 84 degrees C or 0 for none.

Bit2 : 1 for PDN1 is true or 0 for none.

Bit3 : 1 for PDN1 is false or 0 for none.

Bit4 : 1 for NoAuto is true or 0 for none.

Bit4 : 1 for NoAuto is false or 0 for none.

List of getting bits from least significant bit as binary data is followings.

Bit0 : 0 for ICT is 84 degrees C or 1 for 30 degrees C.

Bit1 : 0 for PDN1 is false or 1 for PDN1 is true.

Bit2: 0 for NoAuto is false or 1 for NoAuto is true.

GetVR(n) Function [Single]

n: The serial number of the board that starts from zero.

It returns the value of potentiometer as Single-type data from 0 to 100. The unit is percent. The potentiometer is for display and does not affect the circuit.

InitialPG1(n) Property [Boolean]

n: The serial number of the board that starts from zero.

It sets or gets the status of PG1 when powering on.

True : PG1 is On when the powering on.

False : PG1 is Off when the powering on.

InitialPG2(n) Property [Boolean]

n: The serial number of the board that starts from zero.

It sets or gets the status of PG2 when powering on.

True : PG2 is On when the powering on.

False : PG2 is Off when the powering on.

It will be external trigger mode when powering on if both InitialPG1 and InitialPG2 is off.

InitialLDON(n) Property [Boolean]

n: The serial number of the board that starts from zero.

It sets or gets the status of LD when powering on.

True : LD is On when the powering on.

False : LD is Off when the powering on.

InitialTECON(n) Property [Boolean]

n: The serial number of the board that starts from zero.

It sets or gets the status of TEC when powering on.

True : TEC is On when the powering on.

False : TEC is Off when the powering on.

InitialLongShort(n) Property [Boolean]

n: The serial number of the board that starts from zero.

It sets or gets the pulse width status when powering on. It works for the only board which has pulse width switching circuit. The Long pulse means the pulse width equals trigger pulse width.

True : Long when the powering on.

False : Short when the powering on.

CheckUSB Function

It updates information of USB connection. When the USB connection is changed ,it updates information which is includes DevCount and CardNo.

PulseWidth(n, add, dat) Function [Byte Array]

n: The serial number of the board that starts from zero.

num : [Integer] Selection of pulse width memory address from 0 to 9.

dat : [UShort] Value of pulse width from 10 to 1023.

The actual pulse width is dat * 10 Pico second. The hardware has ten EEPROM address for pulse width. It presets to hardware and write to EEPROM. In case of writing only to EEPROM without presetting to hardware, the address must be added 128. In case of reading from EEPROM without presetting to hardware, the address must be added 128 and dat must be zero.

It returns Byte-type array as followings.

Byte 1 : The address from 0 to 9

Byte 2 : Upper 2 bits out of 10 bits of dat.

Byte 3 : Lower 8 bits out of 10 bits of dat.

dat = Upper 2 bits * 256 + Lower 8bits

Constructor of tmHIDLD

tmHIDLD(MkWinProc)

MkWinProc (Boolean)

True : Use Win32 window message.

False : Not use Win32 window message.

In case using tmDEVICECHANGE from Win32 window message, MkWinProc must be true. The user program catches tmDEVICECHANGE event and uses CheckUSB to update information of hardware. In case not using event, MkWinProc must be false. And the user program reads every certain period of time from Updates property to get timing of connect and disconnect event of USB cable.